

Backend Development — Important Interview Topics

Key concepts to know across APIs, databases, system design, and infrastructure

APIs & Web Fundamentals

RESTful API design

Resource-based URLs, proper HTTP methods (GET/POST/PUT/PATCH/DELETE), status codes, statelessness, and versioning strategies.

HTTP fundamentals

Request/response structure, headers, methods, status code categories (2xx/3xx/4xx/5xx), and the difference between PUT and PATCH.

Authentication vs. authorization

Authentication verifies identity (who you are); authorization determines permissions (what you can do). Know session-based auth, JWT, OAuth 2.0, and API keys.

GraphQL vs. REST

GraphQL lets clients request exactly the fields they need in one call, reducing over/under-fetching, at the cost of more complex caching and server-side query resolution.

Rate limiting & API security

Techniques like token bucket/sliding window rate limiting, input validation, CORS, and protecting against injection attacks and abuse.

Databases

SQL vs. NoSQL

SQL databases (Postgres, MySQL) are relational, schema-enforced, and ACID-compliant; NoSQL (MongoDB, Redis, Cassandra) trades some consistency guarantees for flexible schemas and horizontal scalability.

Database indexing

Indexes speed up read queries by avoiding full table scans, at the cost of slower writes and extra storage — know when and what to index (e.g. B-tree indexes on frequently filtered/joined columns).

Normalization vs. denormalization

Normalization reduces data redundancy via related tables (3NF, etc.); denormalization trades some redundancy for faster reads by avoiding joins — common in read-heavy systems.

Transactions & ACID properties

Atomicity, Consistency, Isolation, Durability — the guarantees a database transaction provides, and how isolation levels (read committed, serializable, etc.) trade off consistency for performance.

Database replication & sharding

Replication copies data across nodes for availability/read scaling; sharding partitions data across nodes to scale writes and storage — know the trade-offs and common sharding strategies.

System Design Fundamentals

Load balancing

Distributing incoming traffic across multiple servers using strategies like round-robin, least connections, or consistent hashing, to improve availability and throughput.

Caching strategies

Where to cache (client, CDN, application, database) and cache invalidation approaches (TTL, write-through, cache-aside) — know the trade-offs of each.

Horizontal vs. vertical scaling

Vertical scaling adds resources to a single server (simpler but has a ceiling); horizontal scaling adds more servers (more complex, requires statelessness, but scales further).

Message queues & async processing

Using queues (RabbitMQ, Kafka, SQS) to decouple services, handle traffic spikes, and process work asynchronously without blocking the main request path.

CAP theorem

A distributed system can only guarantee two of Consistency, Availability, and Partition tolerance at once — understand how this shapes database and system design trade-offs.

Microservices vs. monolith

Monoliths are simpler to develop and deploy initially; microservices offer independent scaling and deployment per service but add operational complexity, network overhead, and distributed-systems challenges.

Server-Side Concepts

Concurrency & parallelism

Concurrency is managing multiple tasks making progress (not necessarily simultaneously); parallelism is literally executing multiple tasks at the same time — know how your language/runtime handles threads or async I/O.

Idempotency

An idempotent operation produces the same result no matter how many times it's performed — critical for safe retries in distributed systems (e.g. PUT requests, payment processing).

Error handling & logging

Structured error responses, meaningful status codes, centralized logging, and monitoring/alerting to detect and diagnose issues in production.

Dependency injection & design patterns

Understanding common patterns (singleton, factory, repository, dependency injection) and how they promote testability and loose coupling in backend code.

Environment configuration & secrets management

Separating config from code (12-factor app principles), using environment variables, and securely managing secrets/credentials rather than hardcoding them.

Testing, Deployment & Reliability

Testing pyramid

Unit tests (fast, isolated, most numerous), integration tests (verify components work together), and end-to-end tests (fewest, slowest, verify full workflows).

CI/CD pipelines

Automating build, test, and deployment stages so code changes are validated and shipped reliably and frequently, reducing manual deployment risk.

Containers & orchestration

Docker packages an app with its dependencies for consistent environments; Kubernetes orchestrates containers at scale, handling deployment, scaling, and failover.

Monitoring & observability

Metrics, logs, and distributed tracing to understand system health and diagnose issues — know the difference between monitoring (known failure modes) and observability (understanding unknowns).

Graceful degradation & fault tolerance

Designing systems to handle partial failures gracefully — circuit breakers, retries with backoff, timeouts, and fallback responses instead of full outages.